

# ui.stepper 全面详解

ui.stepper 是 NiceGUI 基于 Quasar 的 QStepper 组件封装的分步导航元素，用于实现多步骤流程的可视化交互，支持固定步骤配置与动态步骤扩展，广泛适用于向导式操作、流程化任务等场景（如表单提交、安装流程、烹饪步骤引导等）。其核心特性包括步骤切换、状态保持、动态扩展等，且提供丰富的配置项与方法以满足多样化需求。

## 一、核心概念与基础特性

### 1. 本质与用途

- 本质：封装 Quasar 的 QStepper 组件，通过嵌套 `ui.step` 子元素定义单个步骤，每个步骤可包含独立内容与导航控件。
- 核心用途：将复杂流程拆分为有序步骤，引导用户逐步完成操作，提升交互体验与流程清晰度。
- 关键机制：默认使用 Vue 的 `keep-alive` 组件缓存步骤内容，避免切换步骤时动态元素重新渲染导致的问题；若存在客户端性能瓶颈，可手动禁用该特性。

### 2. 基础结构

一个完整的 ui.stepper 由容器（`ui.stepper()`）、步骤（`ui.step()`）、步骤内容（如文本、表单）和导航控件（如“下一步”“上一步”按钮）组成，示例结构如下：

```
from nicegui import ui

with ui.stepper() as stepper: # 步骤容器
    with ui.step('步骤1名称'): # 单个步骤
        ui.label('步骤1内容') # 步骤内展示内容
        with ui.stepper_navigation(): # 导航控件容器
            ui.button('下一步', on_click=stepper.next) # 切换到下一步
    with ui.step('步骤2名称'):
        ui.label('步骤2内容')
        with ui.stepper_navigation():
            ui.button('上一步', on_click=stepper.previous) # 返回上一步
            ui.button('完成', on_click=lambda: ui.notify('流程结束'))

ui.run()
```

## 二、初始化配置项

初始化 `ui.stepper()` 时可通过参数配置核心行为，参数说明如下：

| 参数名             | 类型                 | 说明                                               |
|-----------------|--------------------|--------------------------------------------------|
| value           | ui.step 实例或步骤名称字符串 | 初始选中的步骤（默认值为 None，即选中第一个步骤）                      |
| on_value_change | Callable           | 步骤切换时触发的回调函数（接收步骤变更事件参数）                         |
| keep_alive      | bool               | 是否启用 Vue 的 keep-alive 缓存步骤内容（默认 True，禁用设为 False） |

## 配置示例

```
# 垂直排列的步骤条，初始选中“Ingredients”步骤，切换时触发回调
def on_step_change(e):
    ui.notify(f'当前步骤: {e.value.name}')

with ui.stepper(value='Ingredients', on_value_change=on_step_change,
keep_alive=False).props('vertical').classes('w-full') as stepper:
    with ui.step('Preheat'):
        ui.label('预热烤箱至350华氏度')
    with ui.stepper_navigation():
        ui.button('下一步', on_click=stepper.next)
    with ui.step('Ingredients'):
        ui.label('混合食材')
    with ui.stepper_navigation():
        ui.button('上一步', on_click=stepper.previous).props('flat')
        ui.button('下一步', on_click=stepper.next)
```

## 三、核心属性

ui.stepper 继承 NiceGUI 基础元素的通用属性，支持样式、类名、绑定等配置，关键属性如下：

| 属性名         | 类型               | 说明                                                                        |
|-------------|------------------|---------------------------------------------------------------------------|
| classes     | str              | 元素的 CSS 类名（支持 Tailwind、Quasar 类，如 <code>w-full</code> 占满宽度）               |
| props       | str              | Quasar 组件属性（如 <code>vertical</code> 设为垂直排列， <code>horizontal</code> 水平排列） |
| style       | str              | 内联 CSS 样式（如 <code>color: red;</code> ）                                    |
| value       | BindableProperty | 当前选中的步骤（可通过 <code>bind_value</code> 绑定到外部变量，支持双向同步）                       |
| visible     | BindableProperty | 元素可见性（布尔值，支持动态绑定）                                                         |
| html_id     | str              | HTML DOM 中的元素 ID（版本 2.16.0 新增）                                            |
| is_deleted  | bool             | 元素是否已被删除（只读）                                                              |
| parent_slot | Slot   None      | 父容器的插槽（可设置）                                                               |

## 属性使用示例

```
# 水平排列、红色文字、占满宽度的步骤条
with ui.stepper().props('horizontal').classes('w-full').style('color: red;') as stepper:
    with ui.step('Step 1'):
        ui.label('红色文字的步骤内容')
    with ui.stepper_navigation():
        ui.button('下一步', on_click=stepper.next)
```

## 四、核心方法

ui.stepper 提供丰富的方法用于控制步骤切换、元素管理、绑定等操作，常用方法分类如下：

### 1. 步骤控制方法

| 方法名              | 作用                           | 示例                                      |
|------------------|------------------------------|-----------------------------------------|
| next()           | 切换到下一个步骤                     | <code>stepper.next()</code>             |
| previous()       | 切换到上一个步骤                     | <code>stepper.previous()</code>         |
| set_value(value) | 设置当前选中步骤（值为 ui.step 实例或步骤名称） | <code>stepper.set_value('extra')</code> |

### 2. 元素管理方法

| 方法名                                                            | 作用                    | 参数说明                                                                        |
|----------------------------------------------------------------|-----------------------|-----------------------------------------------------------------------------|
| clear()                                                        | 删除所有子步骤               | 无                                                                           |
| remove(element)                                                | 删除指定子步骤               | element: 子步骤实例或其 ID                                                         |
| move(target_container=None, target_index=-1, target_slot=None) | 移动当前元素到其他容器           | target_container: 目标容器（默认父容器）；target_index: 目标索引（默认追加到末尾）；target_slot: 目标插槽 |
| delete()                                                       | 删除当前 stepper 元素及所有子元素 | 无                                                                           |

### 3. 绑定方法

支持将步骤选中状态、可见性与外部变量绑定，支持单向 / 双向绑定，核心方法如下：

| 方法名                                            | 作用                      | 关键参数                                      |
|------------------------------------------------|-------------------------|-------------------------------------------|
| bind_value(target_object, target_name='value') | 双向绑定选中步骤到目标对象的属性        | target_object: 绑定目标对象；target_name: 绑定的属性名 |
| bind_value_from(...)                           | 单向绑定（从目标对象同步到 stepper）  | 同 bind_value，仅单向同步                        |
| bind_value_to(...)                             | 单向绑定（从 stepper 同步到目标对象） | 同 bind_value，仅单向同步                        |
| bind_visibility(...)                           | 双向绑定可见性到目标对象的属性         | value: 可选，指定目标值匹配时才显示                     |

## 4. 其他常用方法

| 方法名                           | 作用                | 示例                                                                |
|-------------------------------|-------------------|-------------------------------------------------------------------|
| tooltip(text)                 | 为步骤条添加 tooltip 提示 | <code>stepper.tooltip('分步完成操作')</code>                            |
| update()                      | 强制更新客户端的元素状态      | <code>stepper.update()</code>                                     |
| add_resource(path)            | 添加资源文件 (如 CSS、JS) | <code>stepper.add_resource('./static')</code>                     |
| ancestors(include_self=False) | 迭代获取所有祖先元素        | 遍历祖先元素: <code>for elem in stepper.ancestors(): print(elem)</code> |

## 方法使用示例

```
from nicegui import ui

# 动态添加步骤并调整位置
def add_extra_step():
    if len(stepper.default_slot.children) == 2: # 若当前只有2个步骤
        with stepper:
            with ui.step('Extra Step') as extra_step:
                ui.label('动态添加的额外步骤')
                with ui.stepper_navigation():
                    ui.button('上一步', on_click=stepper.previous).props('flat')
                    ui.button('下一步', on_click=stepper.next)
            extra_step.move(target_index=1) # 移动到第2个位置（索引1）

with ui.stepper().props('vertical').classes('w-full') as stepper:
    with ui.step('Start'):
        ui.label('开始步骤')
        ui.button('添加额外步骤', on_click=add_extra_step)
    with ui.stepper_navigation():
        ui.button('下一步', on_click=stepper.next)
    with ui.step('Finish'):
        ui.label('结束步骤')
    with ui.stepper_navigation():
        ui.button('上一步', on_click=stepper.previous).props('flat')

ui.run()
```

## 五、动态步骤扩展

ui.stepper 支持根据业务逻辑动态添加、删除或调整步骤位置，核心依赖 `ui.step()` 实例的 `move()` 方法与 `stepper` 的子元素管理方法，典型场景如下：

## 场景 1：条件性添加步骤

根据用户选择（如复选框、下拉框）决定是否添加额外步骤，示例代码：

```
from nicegui import ui

def next_step():
    # 若勾选“额外步骤”且当前步骤数为2，添加额外步骤并插入到第2位
    if extra_step_checkbox.value and len(stepper.default_slot.children) == 2:
        with stepper:
            with ui.step('Extra') as extra:
                ui.label('这是条件触发的额外步骤')
                with ui.stepper_navigation():
                    ui.button('Back', on_click=stepper.previous).props('flat')
                    ui.button('Next', on_click=stepper.next)
                extra.move(target_index=1) # 插入到“start”和“finish”之间
            stepper.next()

with ui.stepper().props('vertical').classes('w-full') as stepper:
    with ui.step('start'):
        ui.label('开始步骤')
        extra_step_checkbox = ui.checkbox('需要额外步骤? ')
    with ui.stepper_navigation():
        ui.button('Next', on_click=next_step)
    with ui.step('finish'):
        ui.label('结束步骤')
    with ui.stepper_navigation():
        ui.button('Back', on_click=stepper.previous).props('flat')

ui.run()
```

## 场景 2：动态删除步骤

通过 `remove()` 方法删除指定步骤，示例：

```
def delete_step():
    # 删除名为“Extra”的步骤
    for child in stepper.default_slot.children:
        if child.name == 'Extra':
            stepper.remove(child)
            break

# 在步骤中添加“删除额外步骤”按钮
with ui.step('Extra'):
    ui.label('可删除的额外步骤')
    with ui.stepper_navigation():
        ui.button('删除该步骤', on_click=delete_step).props('flat')
        ui.button('下一步', on_click=stepper.next)
```

## 六、事件处理

### 1. 核心事件：步骤切换事件

通过 `on_value_change` 配置步骤切换时的回调，回调函数可接收 `ValueChangeEventArguments` 参数，包含当前选中步骤的信息：

```
def handle_step_change(e):
    # e.value 为当前选中的 ui.step 实例
    ui.notify(f'步骤切换至: {e.value.name} (索引: {stepper.default_slot.children.index(e.value)})')

with ui.stepper(on_value_change=handle_step_change) as stepper:
    # 步骤定义...
```

### 2. 通用事件绑定

通过 `on()` 方法绑定通用 DOM 事件（如点击、鼠标悬浮），支持 Python 回调或 JavaScript 回调：

```
# 绑定点击事件（Python 回调）
stepper.on('click', lambda e: ui.notify('点击了步骤条'))

# 绑定鼠标悬浮事件（JavaScript 回调）
stepper.on('mouseover', js_handler=(e) => console.log("鼠标悬浮: ", e))
```

## 七、高级用法

### 1. 步骤条样式自定义

通过 `props` 和 `classes` 结合 Quasar、Tailwind 类自定义样式，示例：

```
# 垂直排列、紧凑样式、蓝色步骤指示器的步骤条
with ui.stepper().props('vertical dense color=blue').classes('w-1/2 mx-auto') as stepper:
    with ui.step('Step 1'):
        ui.label('自定义样式的步骤条')
        with ui.stepper_navigation():
            ui.button('下一步', on_click=stepper.next).props('color=blue')
```

### 2. 步骤内容缓存控制

默认启用 `keep_alive=True` 缓存步骤内容（如输入框值、动态加载的内容），若需禁用缓存（如步骤内容无需保留状态），可在初始化时设置：

```
# 禁用步骤缓存，切换步骤时重新渲染内容
with ui.stepper(keep_alive=False) as stepper:
    with ui.step('Step 1'):
        ui.input('输入内容（切换步骤后丢失）')
        with ui.stepper_navigation():
            ui.button('下一步', on_click=stepper.next)
```

### 3. 双向绑定步骤状态

将步骤选中状态与外部变量绑定，实现状态同步：

```
from nicegui import ui

class StepState:
    current_step = None

state = StepState()

with ui.stepper() as stepper:
    stepper.bind_value(state, 'current_step') # 双向绑定到 state.current_step
    with ui.step('Step 1') as step1:
        ui.label('步骤1')
        with ui.stepper_navigation():
            ui.button('下一步', on_click=stepper.next)
    with ui.step('Step 2') as step2:
        ui.label('步骤2')
        with ui.stepper_navigation():
            ui.button('上一步', on_click=stepper.previous)

# 外部按钮控制步骤切换（通过修改绑定变量）
ui.button('跳转到步骤1', on_click=lambda: setattr(state, 'current_step', step1))
ui.button('跳转到步骤2', on_click=lambda: setattr(state, 'current_step', step2))

ui.run()
```

## 八、注意事项

- 动态步骤位置调整：使用 `move()` 方法时，`target_index` 基于当前子步骤列表的索引（从 0 开始），需确保索引不越界。
- 性能优化：若步骤包含大量动态元素（如复杂表单、图表），且切换频繁，可禁用 `keep_alive` 以减少内存占用。
- 事件冲突：避免在步骤内容中绑定与 `next()` / `previous()` 冲突的事件（如全局点击事件），防止步骤切换异常。
- 版本兼容性：`html_id` 属性需 NiceGUI 2.16.0+，`bind_value` 的 `strict` 参数需 3.0.0+，使用时需确认版本匹配。

通过以上配置与方法，`ui.stepper` 可灵活满足从简单分步导航到复杂动态流程的各类需求，是 NiceGUI 中实现流程化交互的核心组件之一。